



PRD · Exodus Ops Platform (EOP)

Document ID: PRD-EOP-2077-001

Status: Approved — In Development

Product: Exodus Ops Platform (EOP)

Client: Vault-Tec Reconstruction Division

Engineering: Exodus Systems Inc. — Engineering Division

Cycle: Q1-Q2 · 16 Weeks · 4 Squads (3 engineers each)

Classification: Internal — Engineering Only

Table of Contents

1. [Product Overview](#)
 2. [User Personas](#)
 3. [v0.1 — EOP v0.1 · Core Communication Layer](#)
 4. [v0.2 — EOP v0.2 · HPC Processing Engine](#)
 5. [v0.3 — EOP v0.3 · Public API & Safe Delivery](#)
 6. [v1.0 — EOP v1.0 · Edge Integration & Product Delivery](#)
 7. [Non-Functional Requirements](#)
 8. [Proto Negotiation — Inter-Squad Contract](#)
 9. [Engineering Contract](#)
-

1. Product Overview

1.1 Problem Statement

Vault-Tec Reconstruction Division operates a distributed critical infrastructure composed of multiple bunker nodes connected over a private network. The current state presents five unresolved operational problems:

#	Problem	Operational Impact	Current Baseline
1	No reliable inter-node communication	Bunkers cannot coordinate. Commands are lost. Network state is inconsistent.	MTTR > 4 hours
2	Sequential, single-threaded data processing	Network topology analysis takes hours. Operational decisions arrive too late.	< 1K nodes/min
3	No public API for external integration	AI and corporate services cannot query or control the network.	0 active integrations
4	Edge devices disconnected from the system	Sensors in remote bunkers do not report telemetry to the central system.	0% edge coverage
5	No system observability	Fault diagnosis is manual. No metrics, no automated alerts.	MTTI > 60 min

1.2 Product Vision

EOP is a cloud-native distributed systems platform that enables Vault-Tec to coordinate, process, and observe its distributed infrastructure in real time, with high availability, parallel computation capabilities, and integration with external services and edge devices.

1.3 Goals & Non-Goals

In Scope — EOP v1.0 delivers:

- C++ TCP server with socket-level IPC, supporting N concurrent clients (N > 10k)
- C client library encapsulating the communication protocol
- Parallel HPC processing engine (OpenMP) for network graph analysis
- Go REST API gateway with resilience patterns (circuit breaker, retries, timeouts)
- Edge telemetry ingestion via MQTT + ZephyrOS on ESP32
- Full observability stack: OTel distributed traces, SLO/SLA dashboard, centralized logs
- Kubernetes deployment with canary release and blue-green delivery

Out of Scope — Not built in v1.0:

- Graphical user interface (GUI / visual dashboard)
- mTLS mutual authentication between internal services
- ML/AI for predictive failure detection
- Multi-region or multi-cloud orchestration
- Distributed consensus algorithms in code (Raft, Paxos)
- Legacy hardware support (pre-2015)
- Multi-tenant management with granular roles

1.4 Success Metrics

Metric	Definition	Baseline	Target (EOP v1.0)
MTTR (North Star)	Time from node failure to automatic recovery	> 4 hours	< 200 ms
Manual Intervention Rate	% of incidents resolved without human action	~0% automated	> 80%
Service Availability	Measured by Prometheus probe every 30s	Not measured	≥ 99.9% monthly
Monthly Error Budget	Remaining error budget at release time	Not measured	> 0 at delivery
REST API P99 Latency	End-to-end latency to external client	Not measured	< 300 ms
Edge E2E Latency	ESP32 → EOP server, measured via OTel	Not measured	< 500 ms P99

2. User Personas

Persona	Role & Context	Primary Goal	Current Pain Point
Vault Operator	Operations engineer at Vault-Tec. Works from a Linux terminal. Manages multiple bunkers simultaneously.	Detect and resolve node failures without manual intervention.	Manual diagnosis takes hours. No centralized network state visibility.
Network Analyst	Infrastructure analyst. Needs to process large-scale network topologies to detect bottlenecks and critical nodes.	Run graph analysis on the bunker network and receive results in reasonable time.	Sequential processing takes hours on 50K+ node graphs.
Integration Engineer	Integration engineer at Vault-Tec. Connects external services (AI, corporate systems) to EOP via API.	Consume network state and send commands from external systems reliably and observably.	No public API exists. Integrations are done manually with direct node access.
Edge Device	ESP32 running ZephyrOS in remote bunkers. Intermittent connectivity.	Report telemetry to the central system reliably, even under poor connectivity.	Sensors are disconnected from the system. Data is lost.

3. Milestone 1 — EOP v0.1 · Core Communication Layer

Release Goal: A Vault Operator can register bunker nodes, send commands across the network, and receive responses with basic system observability — all without manual intervention in the communication layer.

Squad scope: 3 engineers, ~4 weeks. Each squad has full flexibility to define service granularity. The protocol contract is negotiated collectively at the Proto Negotiation Meeting (Week 2).

Architectural decision — Sockets as IPC mechanism:

The C++ server uses BSD/POSIX sockets directly (`socket`, `bind`, `listen`, `accept`, `recv`, `send`). No RPC framework abstracts the file descriptor. C clients use the same socket API. This is intentional: v0.1's core objective is for engineers to master the communication model at the transport layer — file descriptor management, concurrency over sockets, message framing, and network error handling. The message protocol (types, framing, serialization) is designed by each squad and documented in an ADR. The inter-squad contract is negotiated at the Proto Negotiation Meeting.

Epic E1 — C++ TCP Server

US-101 · Node connection and welcome handshake

As a Vault Operator,

I want to connect to the EOP server from any network node using the C client,

So that I can send commands and receive responses without knowing the server implementation details.

Acceptance Criteria:

- **AC1.** The server listens on a TCP port configurable via `EOP_PORT` environment variable (default: `9026`). The server fails to start with a descriptive error message if the port is already in use.
- **AC2.** A C client can establish a connection using `connect()` to `host:port` and receive a welcome message within 500 ms of connection establishment.
- **AC3.** The welcome message includes: server version, server timestamp, and an assigned session ID for the connection.
- **AC4.** The connection remains open until the client closes it explicitly or the configurable idle timeout expires (default: 30 s, configurable via `EOP_IDLE_TIMEOUT`).
- **AC5.** If the server is unavailable, the client receives a clear error — no crash, no indefinite hang, no silent failure.
- **AC6.** Host and port are the only required parameters to connect. No additional configuration is needed on the client side.

- **AC7.** The server logs each new connection with: timestamp, client IP, assigned session ID, and total active connections count.
 - **AC8.** An integration test verifies the full connect → handshake → disconnect cycle passes with zero errors under repeated execution (100 iterations).
-

US-102 · Concurrent multi-client support

As a Vault Operator,

I want to have multiple clients connected simultaneously without any of them experiencing degraded response times,

So that I can operate several bunkers in parallel from different terminals.

Acceptance Criteria:

- **AC1.** The server supports a minimum of 100 concurrent clients without timeout or error on any of them.
 - **AC2.** Response latency for any active client does not increase by more than 2× when 100 clients are connected versus 1 client (measured under identical workload).
 - **AC3.** A client that disconnects abruptly (process killed with `SIGKILL`) does not affect any other connected clients — no crashes, no stuck responses, no resource leaks.
 - **AC4.** The server has no file descriptor leaks: verified by comparing `/proc/PID/fd` count before and after 1M connect/disconnect cycles.
 - **AC5.** The concurrency model (thread-per-connection, thread pool, or async `select/epoll`) is documented in an ADR with a justified trade-off analysis.
 - **AC6.** ThreadSanitizer reports zero data races when running the concurrent client test suite.
 - **AC7.** The server enforces a configurable maximum connection limit (`EOP_MAX_CLIENTS`). Connections beyond the limit are rejected with a descriptive error, not silently dropped.
 - **AC8.** A load test with 1000 clients sending 10,000 requests each runs to completion with zero server-side errors or client timeouts.
-

US-103 · Node registration

As a Vault Operator,

I want to register a bunker node with its metadata and receive confirmation,

So that the system maintains a centralized registry of which nodes are active on the network.

Acceptance Criteria:

- **AC1.** The client sends a `REGISTER` message with fields: `node_id` (unique string), `bunker_name`, `ip_address`, and `capacity`.

- **AC2.** The server responds with **ACK** including the assigned **node_id** within 200 ms of receiving the request.
 - **AC3.** If the **node_id** already exists, the server responds with **ERROR_DUPLICATE** and a human-readable description — it does not crash, does not overwrite the existing entry.
 - **AC4.** The registry is maintained in memory (persistence is not required in v0.1). The registry survives the client connection closing.
 - **AC5.** A registered node can be queried by **node_id** using a **QUERY_NODE** message. The response includes all registered metadata.
 - **AC6.** **QUERY_NODE** on a non-existent **node_id** returns **ERROR_NOT_FOUND**, not a server crash or empty response.
 - **AC7.** A **LIST_NODES** message returns all currently registered nodes with their status and metadata.
 - **AC8.** Unit tests cover: successful registration, duplicate registration, query of existing node, query of non-existing node, and list of multiple nodes.
-

US-104 · Node disconnection detection and state update

As a Vault Operator,

I want to be notified when a node goes offline and have its state updated automatically,

So that I do not send commands to offline nodes and I always see accurate network state.

Acceptance Criteria:

- **AC1.** The server detects client disconnection within 10 seconds via periodic heartbeat or TCP keepalive (mechanism documented in ADR).
 - **AC2.** Upon detecting disconnection, the node transitions to **OFFLINE** state in the internal registry.
 - **AC3.** **QUERY_NODE** on an **OFFLINE** node returns the correct state — not a lookup error, not stale **ONLINE** data.
 - **AC4.** The server logs the disconnection event with: timestamp, **node_id**, session ID, and duration of the connection.
 - **AC5.** A node can reconnect and re-register without requiring a server restart. Its state transitions back to **ONLINE** upon successful re-registration.
 - **AC6.** The heartbeat interval is configurable via **EOP_HEARTBEAT_INTERVAL** (default: 5 s). The server tolerates one missed heartbeat before marking the node as **OFFLINE**.
 - **AC7.** An integration test verifies the full cycle: register → establish heartbeat → kill client process → verify **OFFLINE** state → reconnect → verify **ONLINE** state.
-

Epic E2 — C Client Library

US-105 · Connection abstraction

As an Integration Engineer,

I want to use an `eop_connect(host, port)` function that abstracts the TCP connection,

So that I can integrate EOP into my tools without writing raw socket management code.

Acceptance Criteria:

- **AC1.** `eop_connect(host, port)` returns an opaque handle `eop_client_t*` on success.
 - **AC2.** On failure, it returns `NULL` and sets a retrievable error code accessible via `eop_last_error()` with a human-readable description.
 - **AC3.** The function has a configurable connection timeout. It never blocks indefinitely.
 - **AC4.** `eop_client.h` exposes only the public API. Internals (file descriptor, buffers, internal state) are fully hidden behind the opaque handle.
 - **AC5.** The library compiles as both a shared library (`.so`) and a static library (`.a`) from the same source, without code changes.
 - **AC6.** The public header is C99-compatible and usable from both C and C++ without modification.
 - **AC7.** A usage example in `examples/connect_basic.c` compiles and runs successfully against a live server.
 - **AC8.** A unit test verifies that `eop_connect` to an unreachable host fails cleanly within the timeout window, with no resource leaks (verified with AddressSanitizer).
-

US-106 · Command send and response receive

As an Integration Engineer,

I want to send a command and receive the response with a single function call,

So that I can integrate EOP into scripts without manually managing message framing.

Acceptance Criteria:

- **AC1.** `eop_send_command(client, cmd_type, payload, len)` sends a message and blocks the calling thread until a response is received.
- **AC2.** It returns an `eop_response_t` with: `status_code`, `payload`, `payload_len`. The caller is responsible for releasing it via `eop_response_free(response)`.
- **AC3.** If the server closes the connection during the wait, the function returns `EOP_ERR_DISCONNECTED` — no crash, no undefined behavior.
- **AC4.** If the server does not respond within the configured command timeout, the function returns `EOP_ERR_TIMEOUT`.
- **AC5.** The function is thread-safe: multiple threads can invoke it concurrently on the same handle with well-defined behavior.

- **AC6.** Calling `eop_response_free(NULL)` is a safe no-op — no crash.
 - **AC7.** Valgrind `--leak-check=full` reports zero leaks in a test that executes 1,000 sequential `send_command` → `response_free` cycles.
 - **AC8.** Unit tests cover: successful command/response, server disconnection mid-wait, timeout expiry, and null handle error handling.
-

US-107 · Clean disconnection and resource release

As an Integration Engineer,

I want to disconnect from the server and have all resources released automatically,

So that long-running clients do not accumulate file descriptor or memory leaks.

Acceptance Criteria:

- **AC1.** `eop_disconnect(client)` calls `shutdown(SHUT_RDWR)` before `close()` on the socket, ensuring the server receives the `FIN` segment.
 - **AC2.** `eop_disconnect` releases all memory associated with the handle — buffers, internal state, and the handle itself.
 - **AC3.** Calling `eop_disconnect(NULL)` is a safe no-op — no crash, no undefined behavior.
 - **AC4.** After `eop_disconnect`, using the handle is documented as undefined behavior in the public header.
 - **AC5.** AddressSanitizer reports zero errors in a test that runs 1,000 connect → send → disconnect cycles sequentially.
 - **AC6.** The server correctly detects the clean disconnection (via `recv` returning 0) and marks the node `OFFLINE` within the heartbeat window.
 - **AC7.** A concurrent test with 10 threads, each running 100 connect → send → disconnect cycles, completes with zero errors under ThreadSanitizer.
-

Epic E3 — Containerization & Kubernetes

US-108 · Reproducible deployment

As a Vault Operator,

I want to deploy the full EOP stack with a single command on any machine,

So that Vault-Tec can reproduce the environment without manual dependency installation.

Acceptance Criteria:

- **AC1.** The server has a multi-stage `Dockerfile`: a `build` stage with the C++ compiler and a `runtime` stage without it. The runtime image is ≤ 50 MB.
- **AC2.** `ENTRYPOINT` uses JSON array form. The C++ process is PID 1 or runs under a proper init process that propagates `SIGTERM`.

- **AC3.** `SIGTERM` triggers graceful shutdown: the server stops accepting new connections, waits for active connections to finish (maximum 15 s), and exits with code `0`.
 - **AC4.** `docker compose up` brings up the full stack (server + OTel Collector + SigNoz) and all services pass their health checks in under 60 seconds.
 - **AC5.** Kubernetes manifests (`Deployment` + `Service`) are included. `kubectl apply -f k8s/` deploys the server without additional configuration.
 - **AC6.** All configuration is injected via environment variables. No hardcoded values exist in the image or in the K8s manifests themselves (use `ConfigMap`).
 - **AC7.** The image is built in CI on every pull request. A failed build blocks the merge.
 - **AC8.** A `make deploy-local` target in the `Makefile` executes `docker compose up` and runs a smoke test that verifies the server accepts connections.
-

US-109 · Automatic pod recovery

As a Vault Operator,

I want to have the server pod restart automatically if the process crashes,

So that service availability does not depend on someone watching the terminal.

Acceptance Criteria:

- **AC1.** The Pod has `restartPolicy: Always` and a `liveness` probe (TCP check to the server port every 10 s, `failureThreshold: 3`).
 - **AC2.** The Pod has a `readiness` probe (TCP check to the server port). It is marked `NotReady` until the server is ready to accept connections.
 - **AC3.** If the server process crashes, Kubernetes restarts it. Time from crash to Pod `Running` (measured) is < 10 seconds.
 - **AC4.** Pod logs are accessible via `kubectl logs` and include the crash reason if one occurred.
 - **AC5.** A client that has an active connection when the server crashes receives a clean disconnection error — no silent hang.
 - **AC6.** After the Pod restarts, a client can reconnect and re-register without any manual intervention.
 - **AC7.** An integration test verifies the cycle: `kubectl delete pod` → Pod restarts → client reconnects → registers node successfully.
-

Epic E4 — Basic Observability

US-110 · Distributed traces in SigNoz

As a Vault Operator,

I want to see the flow of operations on the server as distributed traces in SigNoz,

So that I can diagnose latency or errors without reviewing logs line by line.

Acceptance Criteria:

- **AC1.** The server emits one OTel span per relevant operation: `REGISTER`, `QUERY_NODE`, `LIST_NODES`, client connection, client disconnection.
 - **AC2.** Each span includes: `operation.name`, `node.id` (if applicable), `duration_ms`, `result` (OK or ERROR), and `session.id`.
 - **AC3.** Spans are visible in SigNoz within 30 seconds of the operation occurring.
 - **AC4.** Server logs are structured JSON to stdout with fields: `timestamp`, `level`, `trace_id`, `span_id`, `node_id`, `event`, `message`.
 - **AC5.** The `trace_id` in the log matches the `trace_id` of the corresponding span in SigNoz, enabling log-to-trace correlation from v0.1.
 - **AC6.** An integration test verifies that a `REGISTER` operation generates exactly one span in SigNoz with the correct `operation.name` and `result`.
 - **AC7.** The OTel Collector is configured as a separate container in `docker-compose.yml`. It exports to SigNoz. No telemetry is lost if SigNoz is temporarily unavailable (buffer configured).
 - **AC8.** A runbook section in the README describes how to navigate from a specific `node_id` in SigNoz to the log lines that correspond to its operations.
-

4. Milestone 2 — EOP v0.2 · HPC Processing Engine

Release Goal: A Network Analyst can request parallel analysis of bunker network topology and receive results in seconds. Service SLOs are defined, measured, and visible in Grafana.

Thread of continuity:

The HPC engine is not a standalone process. It receives network topology data from the v0.1 C++ server via the negotiated socket protocol. The end-to-end flow must work: C client → v0.1 → HPC engine → result back to client. If this flow does not work, v0.2 is not complete.

Epic E5 — Parallel HPC Engine (C++/OpenMP)

US-201 · Parallel graph analysis request

As a Network Analyst,

I want to send a network topology graph to EOP and receive a critical node analysis,

So that I can identify bottlenecks in the Vault-Tec bunker network without processing data manually.

Acceptance Criteria:

- **AC1.** The system accepts network topology graphs in the format defined in the v0.1 protocol (adjacency list or matrix, as specified in the squad's ADR).
 - **AC2.** The engine computes: parallel Dijkstra (shortest path), connected components, and centrality analysis.
 - **AC3.** For graphs up to 10K nodes, results are returned to the requesting client in < 5 seconds with 4 threads.
 - **AC4.** The response includes: algorithm results, graph size (nodes, edges), processing time (ms), and thread count used.
 - **AC5.** The engine logs per request: graph size received, algorithms executed, wall-clock processing time, thread count.
 - **AC6.** Invalid or malformed graph payloads return a structured error response, not a server crash.
 - **AC7.** Unit tests cover: small graph (10 nodes), medium graph (1K nodes), disconnected graph (multiple components), and graph with a single node.
 - **AC8.** An integration test verifies the full end-to-end flow: C client → v0.1 server → HPC engine → result delivered to client.
-

US-202 · OpenMP parallelism and speedup

As a Network Analyst,

I want to have the engine process large graphs in parallel using multiple CPU cores,

So that analysis of the full Vault-Tec network takes seconds, not hours.

Acceptance Criteria:

- **AC1.** The engine uses OpenMP for parallelization. Thread count is configurable via `OMP_NUM_THREADS` without recompilation.
- **AC2.** With 4 threads, speedup is $\geq 2\times$ compared to 1 thread on graphs of $\geq 50K$ nodes (measured and reported in the Benchmark Report).
- **AC3.** With 8 threads, speedup is $\geq 3\times$ compared to 1 thread on graphs of $\geq 100K$ nodes.
- **AC4.** The engine introduces zero data races: ThreadSanitizer reports no errors with any thread configuration from 1 to `$(nproc)`.

- **AC5.** The Benchmark Report is a deliverable: it documents speedup measurements at 1, 2, 4, 8 threads across three graph sizes (10K, 50K, 100K nodes), and includes Amdahl's Law analysis identifying the serial fraction.
 - **AC6.** The Benchmark Report includes a "when not to parallelize" conclusion: the minimum graph size at which parallelization produces positive returns.
 - **AC7.** Benchmark tests are automated and reproducible: `make benchmark` produces the report without manual steps.
 - **AC8.** The engine does not consume more CPU than `OMP_NUM_THREADS` specifies. Verified by observing CPU usage with `top` or `htop` during a benchmark run.
-

US-203 · Integration with v0.1 server

As a Vault Operator,

I want to trigger network analysis through the same interface I use to register nodes,

So that the HPC capability is part of the EOP workflow, not a separate tool.

Acceptance Criteria:

- **AC1.** The HPC engine registers itself with the v0.1 server on startup, using the same `REGISTER` message as any EOP node.
 - **AC2.** A client C sends an `ANALYZE_GRAPH` message to the v0.1 server, which dispatches it to the HPC engine — no direct client-to-engine connection required.
 - **AC3.** The v0.1 server routes the analysis request to the engine and returns the result to the originating client.
 - **AC4.** If the HPC engine is offline, the v0.1 server returns a structured `ERROR_SERVICE_UNAVAILABLE` — no crash, no hang.
 - **AC5.** OTel spans cover the full dispatch path: client request span → server dispatch span → engine processing span. All three are visible and linked in SigNoz.
 - **AC6.** An integration test verifies: engine offline → client receives error → engine comes online → client retries → result delivered.
 - **AC7.** The protocol extension for `ANALYZE_GRAPH` is documented as an addendum to the v0.1 protocol ADR, with the squad's rationale.
 - **AC8.** The integration does not introduce breaking changes to any existing v0.1 messages. `buf breaking` (or protocol diff tool) confirms backward compatibility.
-

Epic E6 — Kubernetes Workload Management

US-204 · Full stack in Kubernetes

As a Vault Operator,

I want to run the complete EOP v0.2 stack in Kubernetes with all services managed by the cluster,

So that I do not need to manage processes manually.

Acceptance Criteria:

- **AC1.** All EOP services (v0.1 server + HPC engine) have K8s `Deployment` manifests with `liveness` and `readiness` probes.
 - **AC2.** All `Deployments` have `resources.requests` and `resources.limits` defined for CPU and memory.
 - **AC3.** `kubectl get pods --all-namespaces` shows all EOP pods in `Running` state after `kubectl apply`.
 - **AC4.** A `ConfigMap` manages all service configuration (ports, timeouts, thread counts). No hardcoded values in container images.
 - **AC5.** The HPC engine can run as a K8s `Job` (for one-shot batch analysis) or as a `Deployment` (for continuous availability). Both modes are documented.
 - **AC6.** A `Makefile` target `make k8s-apply` applies all manifests and waits for all pods to reach `Ready` state, with a timeout of 3 minutes.
 - **AC7.** `kubectl rollout restart deployment/eop-server` completes without any 5xx errors observed on active clients (verified with a concurrent load test).
 - **AC8.** All K8s manifests pass `kubeval` or `kubeconform` validation in CI. Invalid manifests block the merge.
-

Epic E7 — SLO Dashboard & Observability

US-205 · SLO dashboard in Grafana

As a Vault Operator,

I want to see the system SLOs in real time in Grafana,

So that I always know whether EOP is meeting its availability contract with Vault-Tec.

Acceptance Criteria:

- **AC1.** Grafana has a dashboard with: uptime percentage, error budget remaining, P99 latency per service, and requests per second.
- **AC2.** The SLO of $\geq 99.9\%$ availability is displayed as a gauge with clear green/yellow/red thresholds.
- **AC3.** An alert fires when the error budget drops below 50%. The alert is visible in Grafana and in the server logs.
- **AC4.** Prometheus scrapes metrics from all pods via K8s annotations every 15 seconds.
- **AC5.** The dashboard covers a 24-hour window by default, with a configurable time range selector.
- **AC6.** A 5-minute load test with 10 concurrent clients is visible on the dashboard in real time during execution.

- **AC7.** Dashboard definitions are stored as JSON in the repository under `monitoring/grafana/dashboards/`. They are provisioned automatically — no manual import required.
 - **AC8.** The README documents how to access Grafana (URL, default credentials) and how to interpret each panel.
-

US-206 · HPC engine performance panel

As a Network Analyst,

I want to see the HPC engine's processing performance in a dedicated Grafana panel,

So that I can present concrete data to Vault-Tec demonstrating the value of parallel processing.

Acceptance Criteria:

- **AC1.** Grafana has a dedicated HPC panel showing: graphs processed per minute, processing latency (P50, P95, P99), and live speedup ratio vs. single-thread baseline.
 - **AC2.** The panel updates in real time during graph processing.
 - **AC3.** The Benchmark Report from US-202 is referenced in the dashboard documentation as historical evidence.
 - **AC4.** An alert fires if processing P99 latency exceeds 2× the single-thread baseline for more than 2 consecutive minutes.
 - **AC5.** OTel spans from the HPC engine are visible in SigNoz and linked to the originating spans from the v0.1 server (same `trace_id`).
 - **AC6.** The panel shows thread utilization: how many of the configured `OMP_NUM_THREADS` are actively processing at any moment.
 - **AC7.** A `make load-test-hpc` target runs a series of analysis requests and produces a summary report matching the Grafana panel data.
-

5. Milestone 3 — EOP v0.3 · Public API & Safe Delivery

Release Goal: An Integration Engineer can consume network state and trigger operations via a versioned REST API. The team can deploy new versions of EOP without service interruption using canary release and blue-green deployment.

Epic E8 — Go API Gateway

US-301 · Node state REST API

As an Integration Engineer,

I want to query the state of all network nodes via a documented REST API,

So that I can integrate EOP into external systems without knowing the internal socket protocol.

Acceptance Criteria:

- **AC1.** `GET /v1/nodes` returns a JSON array of all registered nodes with: `node_id`, `bunker_name`, `status`, `ip_address`, `last_seen_at`.
 - **AC2.** `GET /v1/nodes/{node_id}` returns the full detail of a specific node. Returns `404` with a structured error body if the node does not exist.
 - **AC3.** All responses use `Content-Type: application/json`. Response bodies follow a consistent envelope: `{ "data": ..., "meta": { "timestamp": ... } }`.
 - **AC4.** An OpenAPI 3.0 specification is available at `GET /v1/openapi.json` and is a mandatory deliverable of v0.3.
 - **AC5.** The API is versioned under `/v1/`. Breaking changes require `/v2/` — this policy is documented in the ADR.
 - **AC6.** The gateway delegates to the v0.1 C++ server using the existing socket protocol. It does not reimplement business logic.
 - **AC7.** `GET /v1/health` returns `200 OK` with service status. This endpoint is used as the K8s readiness probe.
 - **AC8.** Integration tests cover: list nodes (empty registry), list nodes (populated), get existing node, get non-existing node (404), and health check.
-

US-302 · Graph analysis REST API

As an Integration Engineer,

I want to trigger network topology analysis via REST and receive results,

So that I can drive HPC computations from external systems without knowing the internal protocol.

Acceptance Criteria:

- **AC1.** `POST /v1/graph/analyze` accepts a JSON payload with the graph and requested algorithms. Returns results synchronously for graphs < 1K nodes (within 5 s).
- **AC2.** For graphs $\geq$ 1K nodes, the endpoint returns `202 Accepted` with a `job_id`. `GET /v1/graph/jobs/{job_id}` returns job status and results when complete.

- **AC3.** If the HPC engine is offline, the endpoint returns **503 Service Unavailable** with a descriptive message — the internal error is not exposed.
 - **AC4.** The gateway forwards the request to the v0.1 server via the existing socket protocol. No direct gateway-to-engine connection.
 - **AC5.** Job results are available for at least 5 minutes after completion before being evicted.
 - **AC6.** **GET /v1/graph/jobs** returns the list of recent jobs with their status, creation time, and completion time.
 - **AC7.** Invalid graph payloads (missing fields, wrong types) return **400 Bad Request** with a structured validation error that identifies the offending field.
 - **AC8.** The full async flow is verified in an integration test: **POST analyze** → **202** → poll **GET jobs/{id}** → **200** with results.
-

US-303 · Circuit breaker for external services

As a Vault Operator,

I want to have the API gateway tolerate failures in external services without degrading EOP,
So that a failure in the AI or corporate services does not bring down the platform.

Acceptance Criteria:

- **AC1.** The gateway implements a circuit breaker per external service (AI Python service, Corporate Spring Boot service).
 - **AC2.** The circuit opens after 5 consecutive errors or an error rate > 50% in a 10-second window.
 - **AC3.** While the circuit is open, the gateway returns a predefined fallback response (cached data or a degraded response), not a **500** error.
 - **AC4.** The circuit transitions to **HALF-OPEN** every 30 seconds. It closes automatically if the probe request succeeds.
 - **AC5.** The state of each circuit breaker (**OPEN**, **CLOSED**, **HALF-OPEN**) is exposed as a metric in Prometheus and visible in the Grafana SLO dashboard.
 - **AC6.** The fallback response is documented in the OpenAPI spec as the expected response when the circuit is open.
 - **AC7.** An integration test verifies: external service goes down → circuit opens after threshold → fallback response served → service recovers → circuit closes → normal response served.
 - **AC8.** The circuit breaker configuration (threshold, window, half-open interval) is externalized in a **ConfigMap**. No changes require redeployment.
-

Epic E9 — Canary Release & Blue-Green Deployment

Why this epic exists in the product:

EOP v1.0 must maintain > 99.9% uptime when delivered to Vault-Tec. A naive

deployment (`kubectl set image`) has a downtime window. Engineers learn the delivery patterns that eliminate that window. The objective is not to memorize the concept — it is to execute it live during the v0.3 demo with dashboards showing real traffic split.

US-304 · Canary release of api-gateway

As a Vault Operator,

I want to deploy a new version of the API gateway gradually, without risk to current users,

So that if the new version has a defect, its impact is limited and rollback is immediate.

Acceptance Criteria:

- **AC1.** Two Deployments coexist: `api-gateway-stable` (v1) and `api-gateway-canary` (v2), with an nginx `Ingress` splitting traffic by weight.
 - **AC2.** The canary starts at 10% of traffic. The percentage is adjustable via annotation (`nginx.ingress.kubernetes.io/canary-weight`) without redeploying.
 - **AC3.** SigNoz displays separate metrics for v1 and v2: error rate, P99 latency, and requests per second.
 - **AC4.** If the canary error rate exceeds 1% in a 5-minute window, a documented rollback procedure sets the canary weight to 0 in < 30 seconds.
 - **AC5.** The traffic progression 10% → 50% → 100% is executed manually during the v0.3 demo, with all dashboards visible.
 - **AC6.** Clients do not experience errors during any traffic split change (verified with a concurrent load test running throughout the demo).
 - **AC7.** Both Deployments share the same K8s `Service`. Only the Ingress weight changes to shift traffic.
 - **AC8.** The canary configuration is version-controlled. A `make canary-rollout WEIGHT=10` target and a `make canary-rollback` target automate the weight changes.
-

US-305 · Zero-downtime blue-green deployment

As a Vault Operator,

I want to switch between two complete versions of the API gateway with zero downtime,

So that Vault-Tec experiences no service interruption during EOP deployments.

Acceptance Criteria:

- **AC1.** Two complete Deployments exist in parallel: `blue` (current production) and `green` (new version).
- **AC2.** The K8s `Service` selector points to `blue`. Switching to `green` is done by updating the selector label.
- **AC3.** Automated smoke tests run against `green` before the switch. If any smoke test fails, the switch does not occur and `blue` remains active.

- **AC4.** The service switch produces zero `5xx` errors on active clients (verified with a continuous load test running throughout the switch).
 - **AC5.** Rollback (green → blue) uses the identical selector switch mechanism and completes in < 10 seconds.
 - **AC6.** The smoke test suite is defined in `tests/smoke/` and covers: health check, node listing, and a sample graph analysis request.
 - **AC7.** A `make blue-green-switch` target executes the smoke tests and, if they pass, performs the selector switch atomically.
 - **AC8.** The full blue-green cycle (deploy green → smoke tests → switch → verify → rollback) is demonstrated live in the v0.3 demo with zero client errors observed.
-

Epic E10 — Centralized Logging

US-306 · Unified log access in Loki

As a Vault Operator,

I want to search logs from any EOP service through a unified interface,

So that I do not need to run `kubectl logs` on each pod individually to diagnose a problem.

Acceptance Criteria:

- **AC1.** Loki collects logs from all EOP pods. Grafana Explore allows filtering by service, log level, and `trace_id`.
 - **AC2.** Every log entry includes a `trace_id` field that matches the OTel trace of the request that generated it.
 - **AC3.** Given a span in SigNoz, an operator can navigate to the exact log lines for that request in Grafana Explore using the `trace_id`.
 - **AC4.** Logs are retained for a minimum of 7 days. Logs are not lost during pod restarts.
 - **AC5.** Log collection is automatic: no application-level configuration changes are needed when a new pod starts.
 - **AC6.** An integration test verifies: a failed operation generates a span with `result=ERROR` in SigNoz AND a corresponding error log in Loki with the same `trace_id`.
 - **AC7.** A runbook section describes how to correlate a Grafana alert to the specific request logs that triggered it.
 - **AC8.** Log volume is bounded: the Loki retention policy is stored as code in `monitoring/loki/config.yaml` and enforced automatically.
-

6. Milestone 4 — EOP v1.0 · Edge Integration & Product Delivery

Release Goal: Edge devices in Vault-Tec bunkers send telemetry to the central system reliably. EOP v1.0 is a deliverable product with a demonstrable SLA. The demo to Vault-Tec is the final acceptance gate.

ZephyrOS on ESP32 — Technical scope:

The ESP32 runs ZephyrOS. The firmware implements two ZephyrOS application threads: (1) a sensor thread that samples data every 1 second, and (2) an MQTT client thread that publishes to the broker with QoS 1. The pedagogical objective of the RTOS is the model of concurrent tasks in a resource-constrained environment (512 KB RAM). This is precisely why MQTT and not gRPC: the HTTP/2 overhead is physically unviable on that hardware. Engineers must be able to articulate this trade-off in the final defense.

Epic E11 — ZephyrOS on ESP32 + MQTT Publisher

US-401 · Reliable telemetry publishing from edge device

As an Edge Device,

I want to publish bunker telemetry to the central system every second reliably,

So that the Vault-Tec operations team can monitor remote bunkers in real time.

Acceptance Criteria:

- **AC1.** The ZephyrOS firmware has a sensor thread that samples `node_id`, simulated temperature, and `status` every 1 second.
 - **AC2.** An MQTT client thread publishes samples to the topic `eop/bunker/{node_id}/telemetry` with QoS 1.
 - **AC3.** Publication latency from sampling to broker delivery is < 100 ms under normal connectivity.
 - **AC4.** The payload is JSON: `{ "node_id": "...", "timestamp": "...", "temperature": ..., "status": "..." }`.
 - **AC5.** If the broker is unreachable, the firmware does not crash or enter an infinite retry loop. It logs the error and retries with exponential backoff (1 s, 2 s, 4 s, 8 s, max 30 s).
 - **AC6.** The firmware boots and begins publishing within 10 seconds of power-on.
 - **AC7.** MQTT broker connection parameters (`host`, `port`, `topic_prefix`) are configurable via a ZephyrOS `prj.conf` file — no hardcoded values in source code.
 - **AC8.** A ZephyrOS `shell` command `eop status` prints the current connection state, last published timestamp, and total messages sent since boot.
-

US-402 · Offline buffering and guaranteed delivery

As an Edge Device,

I want to buffer messages locally when I lose WiFi connectivity and deliver them upon reconnection,

So that no telemetry is lost from bunkers during temporary network outages.

Acceptance Criteria:

- **AC1.** When WiFi connectivity is lost, the firmware enqueues messages in a local RAM buffer with capacity for at least 60 messages (60 seconds of data).
 - **AC2.** Upon reconnection, the firmware flushes the buffer before publishing new messages, preserving FIFO order.
 - **AC3.** If the buffer is full, the oldest messages are discarded. This behavior is documented in the firmware README and in the ZephyrOS shell output.
 - **AC4.** A manual test verifies: disable WiFi for 30 seconds → re-enable → all buffered messages arrive at the broker without duplicates.
 - **AC5.** Reconnection uses exponential backoff: 1 s, 2 s, 4 s, 8 s, maximum 30 s between attempts.
 - **AC6.** The MQTT client thread publishes a **RECONNECTED** event to `eop/bunker/{node_id}/events` each time connectivity is restored, with the number of buffered messages delivered.
 - **AC7.** Message deduplication is implemented in the edge ingestor (US-404) using the `message_id` field in the payload. The ESP32 firmware assigns a monotonically increasing `message_id` to each message.
 - **AC8.** Buffer overflow events (messages discarded due to full buffer) are tracked as a metric and visible in the Grafana edge panel.
-

US-403 · ZephyrOS thread resilience

As a Vault Operator,

I want to have the ESP32 firmware recover from individual thread failures without manual intervention,

So that edge devices are resilient and do not require physical access to restart.

Acceptance Criteria:

- **AC1.** Each ZephyrOS thread has an independent watchdog. If a thread does not check in within 5 seconds, the watchdog restarts it.
- **AC2.** A thread restart counter is maintained per thread and published to `eop/bunker/{node_id}/health` on each restart.
- **AC3.** A restart of the MQTT thread does not interrupt the sensor thread (and vice versa). Both threads operate independently.
- **AC4.** If ZephyrOS detects a fatal kernel error, the device reboots automatically and resumes normal operation.

- **AC5.** The ESP32 LED blinks in a distinct pattern when a thread is in an error state (documented in the firmware README as a visual debug indicator).
 - **AC6.** A simulated thread fault (injected in a test build via a Kconfig option) triggers the watchdog and the restart counter increments correctly.
 - **AC7.** Thread restart events are published to the MQTT health topic within 10 seconds of the restart occurring.
-

Epic E12 — Edge Ingestor (Go)

US-404 · MQTT event ingestion and routing

As a Vault Operator,

I want to have edge device events automatically reach the EOP server without manual integration,

So that edge telemetry becomes part of the network state that EOP manages.

Acceptance Criteria:

- **AC1.** The edge ingestor subscribes to `eop/bunker/+/telemetry` on the MQTT broker at startup.
 - **AC2.** Each received message is forwarded to the v0.1 C++ server using the existing socket protocol.
 - **AC3.** The ingestor implements idempotency: messages with a duplicate `message_id` (seen within the last 5 minutes) are acknowledged to the broker but not forwarded to the server.
 - **AC4.** An OTel span covers the path from MQTT message receipt to socket delivery to the v0.1 server. The `trace_id` is propagated in the message metadata.
 - **AC5.** If the v0.1 server is offline, the ingestor retries delivery with backoff and logs each attempt. Messages are not dropped silently.
 - **AC6.** The ingestor reconnects to the MQTT broker automatically if the connection drops, with exponential backoff.
 - **AC7.** `GET /v1/edge/status` on the API gateway returns the ingestor connection state, messages received, and messages forwarded in the last 60 seconds.
 - **AC8.** An integration test verifies: ESP32 simulator publishes 10 messages → ingestor receives all 10 → v0.1 server registers the updates → `GET /v1/nodes` reflects the updated state.
-

US-405 · Edge latency dashboard

As a Vault Operator,

I want to see end-to-end latency from the ESP32 to the EOP server in Grafana,

So that I can verify the edge telemetry pipeline meets its SLO.

Acceptance Criteria:

- **AC1.** The OTel trace for each edge message includes: publish timestamp (from the ESP32 payload), MQTT receipt timestamp at the ingestor, and socket delivery timestamp to the v0.1 server.
 - **AC2.** SigNoz displays E2E latency (ESP32 → v0.1 server) for each message as an individual trace.
 - **AC3.** Grafana has a dedicated edge panel showing: messages per second, E2E latency (P50, P99), and message loss rate (buffer overflows).
 - **AC4.** An alert fires if E2E latency P99 exceeds 500 ms for more than 1 consecutive minute.
 - **AC5.** The edge panel is part of the Vault-Tec delivery dashboard shown in the v1.0 demo.
 - **AC6.** A `make edge-test` target runs a 5-minute synthetic load from an ESP32 simulator and produces a summary matching the Grafana panel data.
 - **AC7.** The panel shows broker connectivity status for each known edge device (connected / disconnected / last seen).
-

Epic E13 — EOP v1.0 Product Delivery

US-406 · Reproducible production deployment

As a Vault Operator,

I want to receive EOP v1.0 as a deployable product that launches with a single command,

So that Vault-Tec can operate the platform without continuous support from the Exodus Systems team.

Acceptance Criteria:

- **AC1.** A Helm chart or kustomize overlay deploys the complete EOP v1.0 stack with `kubectl apply`. No manual steps are required after `apply`.
- **AC2.** The repository README contains verified deployment instructions tested on a clean Kubernetes cluster.
- **AC3.** All production configuration (ports, retention periods, SLO thresholds) is in a dedicated configuration file, separate from application code.
- **AC4.** The full stack reaches `Ready` state in under 3 minutes from `kubectl apply`.
- **AC5.** A `RUNBOOK.md` documents: how to scale individual services, how to perform a rollback, and how to interpret the Grafana SLO dashboard.
- **AC6.** A `CHANGELOG.md` documents all changes from v0.1 through v1.0, organized by release.
- **AC7.** All ADRs (from v0.1 through v1.0) are present in `/docs/adr/`, linked from the README, and current.
- **AC8.** A `make smoke-test-prod` target runs against the deployed stack and verifies: server connectivity, node registration, graph analysis, edge message ingestion, and API gateway health — all pass.

As a Vault Operator,

I want to receive documented evidence that EOP met the 99.9% availability SLA during the testing cycle,

So that I have contractual proof of the platform's reliability.

Acceptance Criteria:

- **AC1.** The Grafana SLO dashboard shows compliance data for the last 30 days with 1-hour granularity.
 - **AC2.** The error budget at delivery time is > 0 (the SLA was not violated during the testing cycle).
 - **AC3.** A dashboard screenshot or export from the testing period is included as a v1.0 deliverable.
 - **AC4.** During the live demo, a pod is deleted with `kubect1 delete pod`. The MTTR (pod crash $\rightarrow$ pod ready) is visible in Grafana and is < 200 ms.
 - **AC5.** During the live demo, a canary release of EOP v1.0 is executed with both the stable and canary dashboards visible, showing real traffic split.
 - **AC6.** During the live demo, an ESP32 message flow is shown end-to-end: device publishes $\rightarrow$ SigNoz trace appears $\rightarrow$ `GET /v1/nodes` reflects the update.
 - **AC7.** The demo is reproducible: a `make demo-script` file documents the exact sequence of commands executed during the presentation.
 - **AC8.** Post-demo, the system continues to operate normally. The demo itself does not leave the cluster in a degraded state.
-

7. Non-Functional Requirements

NFRs are system constraints, not optional features. Violating an NFR blocks the release of the corresponding Lab.

NFR-1: Performance

Requirement	Measurement	Threshold	Enforced From
C++ server response latency (P99)	OTel trace duration	< 200 ms	v0.1 $\rightarrow$
Minimum concurrent TCP clients	Load test with N simultaneous clients	≥ 10 without degradation	v0.1
HPC speedup with 4 threads (50K-node graph)	Benchmark single vs. N threads	$\geq 2\times$	v0.2

Requirement	Measurement	Threshold	Enforced From
HPC speedup with 8 threads (100K-node graph)	Benchmark single vs. N threads	$\geq 3\times$	v0.2
REST API P99 latency (end-to-end)	Prometheus + Grafana	< 300 ms	v0.3 →
Edge E2E latency (ESP32 → v0.1 server, P99)	OTel trace with ingestor span	< 500 ms	v1.0

NFR-2: Reliability

Requirement	Measurement	Threshold	Enforced From
Service availability	Prometheus uptime probe every 30 s	$\geq 99.9\%$ monthly	v0.2 →
MTTR on Pod crash	Grafana: pod death → pod ready time	< 200 ms	v0.2 →
Graceful shutdown on SIGTERM	Exit code 0. No active client errors.	100% of cases	v0.1 →
Zero message loss during offline buffering	WiFi off 30 s → reconnect → count comparison	0 messages lost	v1.0
Canary rollback time	Timed during live demo	< 30 s	v0.3 →
Blue-green switch zero-downtime	5xx count during switch under load test	0 errors	v0.3 →

NFR-3: Code Quality

Requirement	Verification Tool	Severity
RAII for all resources (sockets, fds, memory, mutexes)	Code review + AddressSanitizer	BLOCKING
No memory leaks	AddressSanitizer <code>--leak-check=full</code> in CI	BLOCKING
No data races	ThreadSanitizer on concurrency test suite in CI	BLOCKING
C++ exceptions do not cross module boundaries	Code review. Public interfaces use error codes.	BLOCKING

Requirement	Verification Tool	Severity
Test coverage $\geq$ 80% on business logic	<code>gcov</code> or <code>llvm-cov</code> in CI	Required
Clang-Tidy: no critical warnings (<code>bugprone</code> , <code>concurrency</code>)	<code>clang-tidy</code> in CI — blocks merge	Required
Runtime Docker image $\leq$ 50 MB	<code>docker images</code> check in CI	Required
K8s manifests pass schema validation	<code>kubeval</code> or <code>kubeconform</code> in CI	Required

NFR-4: Operability

- All services expose a health endpoint or K8s liveness probe responding in < 100 ms.
- All logs are structured JSON to stdout with fields: `timestamp`, `level`, `service`, `trace_id`, `span_id`, `message`.
- Every relevant operation emits an OTel span. Spans are visible in SigNoz within 30 seconds.
- The full stack launches with a single command (`docker compose up` or `kubectl apply`).
- Deployment is reproducible: identical manifests produce identical results on any compliant Kubernetes cluster.
- All configuration is externalized. No environment-specific values are hardcoded in source code or container images.

8. Proto Negotiation — Inter-Squad Contract

The multi-squad model

The four squads (Catalyst, Hyperion, Nexus, Orion) are engineering teams building the **same product**. Their services must interoperate. Each squad has full flexibility to define the granularity of its own services — but the message protocol that connects those services is a **shared contract** that all squads negotiate collectively.

Squad	Service Ownership	Protocol Dependency
Catalyst	Defined in the squad's Service Boundaries ADR.	The message protocol agreed upon at the Proto Negotiation Meeting.
Hyperion	Defined in the squad's Service Boundaries ADR.	Idem.

Squad	Service Ownership	Protocol Dependency
Nexus	Defined in the squad's Service Boundaries ADR.	Idem.
Orion	Defined in the squad's Service Boundaries ADR.	Idem.

Proto Negotiation Meeting — Mandatory Gate, Week 2

Before any implementation code is written, all four squads meet to negotiate the message protocol. This meeting has formal deliverables.

Each squad prepares before the meeting:

1. List of operations their service needs to expose (verbs, payloads, expected responses).
2. List of operations they need to consume from other squads' services.
3. A proposed message format: framing (length-prefixed, delimiter-based), field types, encoding (binary, JSON, custom).
4. A draft Service Boundaries ADR: what their service does, what it does not do, and what guarantees it provides.

The meeting produces:

1. **Protocol v1.0:** Message types, fields, framing, encoding. Signed off by all four squads.
2. **Ownership table:** Which squad implements which side of each communication channel.
3. **Compatibility criteria:** What constitutes a breaking vs. non-breaking change.
4. **Proto Freeze date:** After this date, breaking changes require explicit versioning and a re-negotiation session.

Why this process exists:

In production systems, a squad that modifies the wire protocol without coordinating with dependent teams breaks the system for all users. The Proto Negotiation simulates exactly that dynamic: four teams with independent incentives must reach a technical agreement, document it formally, and honor that contract throughout the project. Designing inter-service communication protocols and managing their evolution are core skills in distributed systems engineering.

9. Engineering Contract

3-Step Delivery Cycle

No feature enters implementation without passing through Design. No release ships without Stabilization.

%	Phase	Exit Gate	Mandatory Artifacts
30%	Design	ADRs approved. Interfaces defined. Protocol negotiated (v0.1).	ADR docs in <code>/docs/adr/</code> . Sequence diagram of the protocol.
50%	Implementation	Tests pass in CI. No leaks. No races. Code review approved.	Code + tests. CI green. PR with ≥ 1 approved review.
20%	Stabilization	Integration tests pass. SLOs green. Documentation current.	Updated README. CHANGELOG entry. Release tag in git.

Definition of Done

A story is **Done** when all of the following are true:

1. Code is merged to `main` with CI fully green.
2. Every acceptance criterion is verifiable with an automated test or a reproducible demo.
3. If the story makes an architectural decision, the corresponding ADR is documented in `/docs/adr/`.
4. No memory leaks or data races are introduced by the story's changes (verified by CI sanitizers).
5. The service README reflects the current state: how to build, how to run locally, how to execute the test suite.

Release Gates

No squad advances to the next Lab without passing the gate of the current one.

Gate	Timing	Criterion	Owner
Proto Freeze	End of Week 2	Protocol v1.0 documented and signed off by all four squads.	PL + All squads
v0.1 Release	End of Week 5	US-101 through US-110 Done. CI green. EOP v0.1 deployable.	Squad + PL

Gate	Timing	Criterion	Owner
v0.2 Release	End of Week 9	US-201 through US-206 Done. SLO dashboard active. K8s all green.	Squad + PL
v0.3 Release	End of Week 12	US-301 through US-306 Done. Canary release demonstrated live.	Squad + PL
EOP v1.0 — Final	Week 16 — Vault-Tec Demo	US-401 through US-407 Done. SLA demonstrable. Helm chart deployable.	Squad + Review Panel

— End of Document PRD-EOP-2077-001 —
Exodus Systems Inc. · Engineering Division · Internal Use Only